Generalized Assignment Problem (GAP)

Assignment of sellers to warehouses in first-mile logistics

Lucas Ojeda

1 Introduction

With the growth of e-commerce, logistics companies are trying to lower the costs of the *first mile*: moving the parcels from each seller to a reception point of the network. A small seller gains nothing from scheduling a pickup visit, so the seller takes the parcels to a **warehouse**: a shop of the network that, as a side activity, keeps them until the logistics company collects them in consolidated form.

Today ThunderPack operates in a decentralized way: each seller chooses which warehouse to take the parcels to and may change without notice. This makes inefficient use of the capacity of the network, above all during demand peaks (Black Friday, Hot Sale), and causes complaints and extra costs. ThunderPack is therefore considering a centralized mode, in which each seller is assigned a fixed warehouse, respecting the capacities and minimizing the total distance traveled. The business questions behind this work are:

- **(Q1)** Is the capacity of the warehouse network enough, or does it need to be expanded to serve every seller?
- **(Q2)** Is there an assignment in which the sellers travel a reasonable distance?
- **(Q3)** Is a tool that delivers good solutions within *a few minutes* viable, usable even in real time during demand peaks?

We model the problem as a **Generalized Assignment Problem** (GAP), which is NP-hard. The work had two stages. In the first one we tackled the real scale ($n = 1100$ sellers, $m = 310$ warehouses) with heuristic methods: constructive heuristics, local search operators chained in a VND and two metaheuristics that use it inside (ILS and LNS). Those solutions could only be compared with each other: without a lower bound there was no way of knowing how far from the optimum they were. In the second stage we added lower bounds (the linear and Lagrangian relaxations), heuristics that start from those relaxations and exact methods on the SCIP solver. With them we measured the true distance to the optimum on 57 benchmark instances and on the real one, and we proved the optimum of 36 of them. In particular, the optimum of the real instance is 679.60, and the solver proves it in less than 10 seconds.

2 Description of the model

Let $N = \{1, \dots, n\}$ be the set of the n sellers and $M = \{1, \dots, m\}$ that of the m warehouses. Each warehouse $i \in M$ has capacity b_i. For a seller $j \in N$ and a warehouse $i \in M$, d_{ij} is the demand (units of capacity that j takes up in i) and c_{ij} the cost of assigning j to i (at ThunderPack, the distance that j would travel). We use binary variables $x_{ij} = 1$ if j is assigned to i. The GAP in its general form is:

$$\min \quad \sum_{i \in M} \sum_{j \in N} c_{ij}\, x_{ij} \tag{1}$$

$$\text{s.t.} \quad \sum_{i \in M} x_{ij} = 1 \qquad \forall\, j \in N \tag{2}$$

$$\sum_{j \in N} d_{ij}\, x_{ij} \le b_i \qquad \forall\, i \in M \tag{3}$$

$$x_{ij} \in \{0, 1\} \qquad \forall\, i \in M,\ j \in N.$$

Constraint (2) assigns each seller to exactly one warehouse and (3) respects the capacities. Equivalently, a solution is a collection of disjoint sets $\Gamma_1, \ldots, \Gamma_m \subseteq N$, with Γ_i the sellers assigned to i. In the real instance the demand of a seller does not depend on the warehouse ($d_{ij} = d_j$); in the benchmark ones it does.

Partial solutions and penalty. Depending on the capacities and demands, there may be no feasible assignment that covers every seller. In that case we admit *partial solutions* and penalize each unassigned seller. With $c_{max} = \max_{i \in M,\, j \in N} c_{ij}$ (the largest possible distance) and $p = 3\, c_{max}$, the objective function we evaluate is

$$f(s) \;=\; \sum_{j \text{ assigned to } i(j)} c_{i(j)\,j} \;+\; p \cdot \bigl|\{j \in N :\ j \text{ unassigned}\}\bigr|. \tag{4}$$

The penalty p per unassigned seller makes placing a seller in any warehouse always better than leaving it out, without making infeasible the instances whose total capacity is not enough. Adding a binary variable u_j that indicates whether j is left unassigned, (4) is the objective of an integer model that a solver can solve directly (§4), the **compact model**:

$$\begin{aligned}
\min \quad & \sum_{i \in M} \sum_{j \in N} c_{ij}\, x_{ij} + p \sum_{j \in N} u_j && && (5)\\
\text{s.t.} \quad & \sum_{i \in M} x_{ij} + u_j = 1 && \forall\, j \in N && (6)\\
& \sum_{j \in N} d_{ij}\, x_{ij} \le b_i && \forall\, i \in M &&\\
& x_{ij},\, u_j \in \{0,1\} && \forall\, i \in M,\ j \in N. &&
\end{aligned}$$

Interpretation as an assignment. It is an assignment problem with capacities (*bin packing* with costs): the sellers are items with demand d_{ij}, the warehouses are bins with capacity b_i, and assigning j to i costs c_{ij}. This guides the heuristics (assign first what is closest or most urgent) and the local operators (move or exchange sellers between warehouses). Moreover, choosing which sellers go to a given warehouse is a knapsack problem: that structure is what the lower bounds and the exact methods exploit.

Data model (implementation). The code mirrors the model: `Warehouse` (`id`, `capacity` b_i); `Seller` (`id`, `demand`, the vector of the d_{ij}, and `distances`, a vector of pairs (c_{ij}, i) **sorted by increasing distance**, so that the closest warehouses are tried first); `Solution` (the warehouse of each seller, the sellers of each warehouse and the total cost, with methods `assign()` and `remove()`); and `Instance` (warehouses, sellers, cost matrix and the penalty p; its constructor reads the OR-Library format).

3 Heuristic algorithms

All the methods work on the same `Solution` representation and, except for the tabu search, keep feasibility ((3)) at every step. We write cap_i for the free capacity of warehouse i in the current solution.

3.1 Constructive heuristics

Nearest neighbor. Goes through the sellers in order and assigns each one to its closest feasible warehouse, taking advantage of `distances` being already sorted. If no warehouse has capacity left, the seller is left unassigned. The first sellers take the best warehouses and the last ones may find no room.

Algorithm 1 Constructive heuristic: nearest neighbor

function NEARESTNEIGHBOR(inst)
  $cap_i \leftarrow b_i \ \forall i$; $sol \leftarrow$ empty; $f \leftarrow 0$
  for each seller $j \in N$ in order **do**
    $assigned \leftarrow$ false
    for all $(dist, i)$ in $distances_j$ (increasing) **do**
      if $cap_i \geq d_{ij}$ **then**
        assign j to i; f += $dist$; cap_i −= d_{ij}; $assigned \leftarrow$ true; **break**
      end if
    end for
    if not $assigned$ **then** f += p
    end if
  end for
  return sol
end function

Regret insertion. At each step, for each unassigned seller it computes its best and second-best feasible warehouse according to a *desirability measure* f_{ij} (lower is better); the regret is the difference between both values (or ∞ if only one is feasible). It assigns the seller with the largest regret to its best warehouse. The idea is to serve first whoever loses the most by not taking its best option now, so as to reduce the bad assignments forced at the end. With $f_{ij} = c_{ij}$ it is the classic regret insertion; following Martello and Toth [7] we also tried $f_{ij} = d_{ij}$ (*demand*) and $f_{ij} = d_{ij}/b_i$ (*relative demand*), which fill the warehouses more slowly. On the instances with tight capacity this matters: guided by the cost, the constructive heuristics leave thousands of sellers unassigned (E1). The *best constructive* solution is the best of the four.

Algorithm 2 Constructive heuristic: regret insertion with measure f

function REGRETINSERTION(inst, f)
  $cap_i \leftarrow b_i \ \forall i$; $sol \leftarrow$ empty; every j unassigned
  while some unassigned j has a feasible warehouse **do**
    for each unassigned j: p_j^1 (p_j^2) = value f_{ij} of its best (second-best) feasible warehouse
    $regret_j \leftarrow p_j^2 - p_j^1$ (or ∞ if it has only one feasible warehouse)
    choose $j^\star = \arg\max_j regret_j$; assign it to its best warehouse; update *cap* and the cost
  end while
  penalize the j left unassigned
  return sol
end function

3.2 Local search operators

Relocate multiple. Generalized operator: moves up to q convenient sellers from a warehouse i to another warehouse k (those with the largest saving that fit in the free capacity of k). The case q=1 is the simple relocate: move a single seller j, with cost change $\Delta = c_{kj} - c_{ij}$, accepted if $c_{kj} < c_{ij}$ and $d_{kj} \leq cap_k$. It is one of the two main operators chained by the VND.

Algorithm 3 *Relocate multiple* operator (first improvement)

function RELOCATEMULTIPLE(sol, inst, q)
  for all pair of warehouses (i, k) with $i \neq k$ **do**
    $move \leftarrow$ up to q sellers of Γ_i with $c_{kj} < c_{ij}$, those with the largest saving $c_{ij} - c_{kj}$, that fit in cap_k
    if $move \neq \emptyset$ **then**
      move each $j \in move$ from i to k; update f; **return** true
    end if
  end for
  return false
end function

Swap multiple. Generalized operator: applies up to q successive exchanges (the best feasible pair each time) between a pair of warehouses i_1, i_2. The case q=1 is the simple swap: exchange two sellers $j_1 \in \Gamma_{i_1}$ and $j_2 \in \Gamma_{i_2}$, with cost change $\Delta = (c_{i_2 j_1} + c_{i_1 j_2}) - (c_{i_1 j_1} + c_{i_2 j_2})$, accepted if $\Delta < 0$ and the exchange respects both capacities. It is the other main operator of the VND.

Algorithm 4 *Swap multiple* operator (first improvement)

function SWAPMULTIPLE(sol, inst, q)
  for all pair of warehouses (i_1, i_2) with $i_1 < i_2$ **do**
    $done \leftarrow 0$
    while $done < q$ **and** there is a feasible exchange with $\Delta < 0$ **do**
      apply the pair $j_1 \in \Gamma_{i_1}$, $j_2 \in \Gamma_{i_2}$ with the smallest $\Delta = (c_{i_2 j_1} + c_{i_1 j_2}) - (c_{i_1 j_1} + c_{i_2 j_2})$; update cap, f; $done$ += 1
    end while
    if $done > 0$ **then return** true
    end if
  end for
  return false
end function

Both generalized operators apply a fixed acceptance strategy (they scan the pairs and keep the first one that improves, choosing the best moves within it): they have no first/best variant. That distinction does exist in the *simple versions* (q=1, a single seller / a single pair), which we implemented separately with two acceptance criteria: *first improvement* (applies the first improving move) and *best improvement* (explores the whole neighborhood and applies the move with the largest gain). In E2 (§6) we compare the simple versions (first vs best) against the two generalized operators; the VND chains the latter in first improvement.

Auxiliary operators for tight instances. When capacity is scarce and some sellers are left unassigned, the previous moves are not always enough. We implemented:

- `assign_unassigned`: tries to place each unassigned seller in its closest feasible warehouse.
- `assign_with_ejection`: to place an unassigned seller j in a warehouse k without room, it ejects a seller x from k and relocates it to another warehouse k' with capacity, if the cost balance is favorable.
- *cyclic exchange* (`cyclic_exchange`): rotates three sellers among three warehouses ($j_1 \to i_2$, $j_2 \to i_3$, $j_3 \to i_1$), freeing room through a cyclic rearrangement that *relocate* and *swap* (limited to two warehouses) cannot reach. To keep it tractable, the target warehouses are restricted to the 5 closest ones of each seller and first improvement is applied. It is decisive when capacity is scarce and all the nearby warehouses are full, so that no move between two warehouses can descend.

3.3 Metaheuristics: VND, ILS and LNS

VND (Variable Neighborhood Descent). Chains all the local search neighborhoods: it applies each one until it is exhausted, in a fixed order, and repeats the whole cycle while any of them improved, until reaching a local optimum with respect to every neighborhood. The implemented version goes through, in first improvement: `assign_unassigned`, ejection, the two main operators *relocate multiple* and *swap multiple* (q=1 . . . 10) and the *cyclic exchange* of 3 warehouses. It is the local improvement subroutine used by both metaheuristics.

Algorithm 5 Metaheuristic: VND (Variable Neighborhood Descent)

```
1: function VND(sol, inst)
2:     improved ← true
3:     while improved do
4:         improved ← false
5:         while AssignUnassigned(sol) do improved ← true
6:         end while
7:         while Ejection(sol) do improved ← true
8:         end while
9:         for q = 1 to 10 do
10:            while RelocateMultiple(sol,q) do improved ← true
11:            end while
12:            while SwapMultiple(sol,q) do improved ← true
13:            end while
14:        end for
15:        while CyclicExchange(sol) do improved ← true
16:        end while
17:    end while
18: end function
```

ILS (Iterated Local Search). Starts from the best constructive solution, already improved with VND. At each iteration it copies the best solution and perturbs it: it applies `strength` random feasible moves between warehouses (half *swaps* and half *relocations*) to escape from the local optimum. Then it evaluates it again, applies VND and keeps the best. Parameters: `strength`, seed and time budget, which includes the construction and the VND of the initial solution.

Algorithm 6 Metaheuristic: ILS (Iterated Local Search)

```
1: function ILS(inst, strength, seed, seconds)
2:     best ← BestConstructive; VND(best); evaluate f(best)
3:     while elapsed time < seconds do
4:         s ← best; Perturb(s, strength) [half random swaps, half random relocations]
5:         evaluate f(s); VND(s)
6:         if f(s) < f(best) then best ← s
7:         end if
8:     end while
9:     return best
10: end function
```

LNS (Large Neighborhood Search / *ruin-and-recreate*). Same structure, but with a larger perturbation: at each iteration it unassigns `destroy` random sellers (*ruin*) and lets the VND reinsert and reoptimize them (*recreate*), accepting only if it improves. By rebuilding a part of the solution from scratch, LNS can rearrange assignments that ILS, tied to local moves, does

not reach. Parameters: `destroy`, seed and time budget; it can also start from a given solution (E9).

Algorithm 7 Metaheuristic: LNS (Large Neighborhood Search)

```
function LNS(inst, destroy, seed, seconds)
    best ← BestConstructive (or a given solution);  VND(best);  evaluate f(best)
    while elapsed time < seconds do
        s ← best;  unassign destroy random sellers of s [ruin]
        VND(s);  evaluate f(s) [recreate]
        if f(s) < f(best) then best ← s
        end if
    end while
    return best
end function
```

3.4 Tabu search

On the tightest instances every improvement first needs to free room in a full warehouse, and a search that never leaves the feasible region gets stuck. Inspired by Yagiura, Ibaraki and Glover [12], we implemented a tabu search that allows exceeding the capacities for a while, charging a penalty for each unit of excess. That is, it minimizes

$$\tilde{f}(s) \;=\; f(s) + \sum_{i \in M} \alpha_i \max\Big(0,\; \sum_{j \in \Gamma_i} d_{ij} - b_i\Big),$$

where the weight α_i of each warehouse adjusts itself: it grows by 5 % at each iteration in which the warehouse is overloaded, and all of them shrink by 2 % when the solution becomes feasible again. Besides moving and exchanging sellers, it uses *ejection chains*: a seller enters a warehouse and pushes another one out to a third. So as not to undo right away what it did, a seller cannot return for some iterations to the warehouse it left.

4 Lower bounds and exact methods

A **lower bound** LB is a value that no solution can beat. It serves two purposes: measuring the *true gap* of a solution, $(f(s) - LB)/LB$, and discarding parts of the problem that only lead to solutions worse than the best known one. When a solution reaches the bound, it is optimal.

4.1 Linear relaxation and roundings

If *fractions* of sellers may be assigned, that is, if $x_{ij}, u_j \in \{0,1\}$ is replaced by $x_{ij}, u_j \in [0,1]$ in (5), the model becomes a linear program (LP), which is solved quickly and gives a lower bound. Its solution has a useful property (Benders and van Nunen [3]): the LP has $n+m$ constraints, so its basic solution has at most $n+m$ positive variables, and since each seller needs at least one, at most m sellers are split among several warehouses. That gives a simple heuristic, the *LP rounding*: the integral sellers keep their warehouse, the split ones are inserted where they fit and then the VND is applied.

We also tried the rounding of Shmoys and Tardos [11], which from the LP solution x^* builds an assignment with

$$\text{cost} \;\le\; \text{LP bound} \qquad \text{and} \qquad \sum_{j \in \Gamma_i} d_{ij} \;\le\; b_i + \max_{j:\, x^*_{ij} > 0} d_{ij} \quad \forall\, i \in M,$$

that is, with a guaranteed cost but possibly exceeding the capacities. And we tried two families of *cutting planes* (inequalities added to the LP to bring it closer to the integer problem). The

first are the cover inequalities: if a set C of sellers does not fit in warehouse i ($\sum_{j\in C} d_{ij} > b_i$), at most $|C|-1$ of them can go to i,

$$\sum_{j\in C} x_{ij} \;\le\; |C| - 1$$

(in the implementation, extended to the sellers heavier than those of C). The second is an exact separation, which looks for the most violated valid inequality for the knapsack of each warehouse (Avella, Boccia and Vasilyev [1]).

4.2 Lagrangian relaxation

The other way of relaxing the problem is to allow a seller to end up in no warehouse or in several, charging it a price λ_j for each violation of (6) (Fisher, Jaikumar and Van Wassenhove [5]). Without that constraint, the problem splits into one knapsack problem per warehouse: which sellers to fill it with, according to their costs and the prices. For each set of prices λ,

$$\begin{aligned} L(\lambda) \;&=\; \sum_{j\in N} \lambda_j \;-\; \sum_{i\in M} KP_i(\lambda) \;+\; \sum_{j\in N} \min(0,\; p - \lambda_j), \\ KP_i(\lambda) \;&=\; \max\Big\{ \sum_{j\in N} (\lambda_j - c_{ij})\, x_j \;:\; \sum_{j\in N} d_{ij}\, x_j \le b_i,\; x \in \{0,1\}^n \Big\}, \end{aligned}$$

is a lower bound. The *subgradient method* keeps adjusting the prices: it raises that of the sellers that ended up in no warehouse and lowers that of those that ended up in several. With $g_j = 1 - \sum_i x_{ij} - u_j$ (the violation of each seller in the solution of the knapsacks) and UB the cost of the best known solution, the step is

$$\lambda \;\leftarrow\; \lambda + t\, g, \qquad t \;=\; \mu\, \frac{UB - L(\lambda)}{\|g\|^2},$$

with μ starting at 2 and halved when the bound stops improving. This bound is much better than the LP one (E7).

The knapsacks are, moreover, almost a solution. The *Lagrangian heuristic* leaves each repeated seller in the cheapest of its warehouses and inserts the missing ones; it turned out to be one of the best starting points we found (E9, E10).

4.3 Variable fixing

With the Lagrangian prices one can compute, for each seller-warehouse pair, a bound on the best possible cost if that seller goes to that warehouse. If KP_i^{+j} and KP_i^{-j} are the knapsack of i with j forced in and forced out, that bound is

$$L_{ij}^1 \;=\; \sum_{l\in N} \lambda_l - KP_i^{+j} - \sum_{k\ne i} KP_k^{-j} + \sum_{l\ne j} \min(0,\; p - \lambda_l),$$

and that of the solutions that do *not* assign j to i is $L_{ij}^0 = L(\lambda) + KP_i - KP_i^{-j}$. If L_{ij}^1 already exceeds the cost UB of the best known solution (with integral costs, if $L_{ij}^1 > UB - 1$), the pair is discarded: no better solution uses it; if L_{ij}^0 is the one that exceeds it, j is fixed to i. All these knapsacks come out of two dynamic programs, one forward and one backward over the sellers, in the time of a single knapsack per warehouse. What is left is a much smaller problem, and the better the known solution, the more pairs are discarded. Since the solver finds better solutions during the search, we also implemented *re-fixing*: each time a better solution appears, the pairs are checked again and the new ones are discarded. Posta, Ferland and Michelon [8] use a similar idea to solve the GAP exactly.

4.4 Exact methods

Integer model in SCIP. We solve the model directly with the SCIP solver [10], giving it the best heuristic solution as a starting point. We tried it alone, after the fixing (on the reduced problem) and with re-fixing during the search.

Branch-and-price. Instead of deciding seller by seller, this formulation chooses for each warehouse a *set* of sellers that fits in it (Savelsbergh [9]). If each column s is a set of sellers that fits in warehouse $i(s)$, with cost $c_s = \sum_{j \in s} c_{i(s)j}$, the problem is

$$\min \sum_s c_s \lambda_s + p \sum_j u_j \quad \text{s.t.} \quad \sum_{s \ni j} \lambda_s + u_j = 1 \;\; (\pi_j), \qquad \sum_{s:\, i(s)=i} \lambda_s \le 1 \;\; (\mu_i), \qquad \lambda_s, u_j \in \{0,1\}.$$

Since there are too many sets to write them all, only the useful ones are generated: with the duals π of the linear program, the best new set of each warehouse is again a knapsack, with profits $\pi_j - c_{ij}$ (*column generation*). At each round, $L(\pi) = \sum_j \min(\pi_j, p) - \sum_i KP_i(\pi)$ is a valid bound, and at the end it matches the best Lagrangian bound. When the solution comes out fractional, it branches, that is, it splits the problem in two (seller j goes to warehouse i, or not). We added the usual techniques to speed it up (stabilization, strong branching, a heuristic on the generated sets, among others) and the fixing.

Bundle method. On the large instances, the branch-and-price does not finish computing the bound of the first node: the linear program that combines the sets is highly degenerate and the simplex moves very slowly. We tried solving it with another linear solver (HiGHS [6], with the simplex and with an interior point method) and with a *bundle method* [4], which computes the same bound without solving that linear program. The method maximizes $L(\pi)$ step by step: at each step it solves, with the sets B_i generated so far,

$$\max_{\pi} \; \sum_j \min(\pi_j, p) - \sum_i \max_{s \in B_i} (\pi(s) - c_s) - \frac{\|\pi - \hat{\pi}\|^2}{2t},$$

where the last term prevents moving far from the best known point $\hat{\pi}$. If the new point improves the bound enough, it becomes $\hat{\pi}$; if not, t is reduced and it tries closer.

4.5 Matheuristic and portfolio

The *MIP-LNS* is an LNS in which the solver does the reconstruction: at each iteration it frees the sellers of a group of nearby warehouses and lets SCIP reassign them in the best possible way (with a limit of 5 s), with the rest fixed. Freeing whole warehouses allows exchanges among them that the VND does not find. The *portfolio* combines everything within the same budget: the LP rounding and the Lagrangian heuristic, then the tabu search from the better of the two and, at the end, the MIP-LNS.

5 Implementation notes and difficulties

- **Build and usage.** Code in C++ (`c++17`). From `src/`, `make` builds `gap_simulator` (ILS on one instance) and the test suite of the heuristics, `./tests <mode>`, with no external dependencies. The exact part uses SCIP and HiGHS, installed with `pixi install`; `make exact` builds `gap_exact`, which runs any method on one instance (e.g., `./gap_exact mipfix instances/real/real_instance 60 <output>` writes the optimal assignment of the real one), and `exact_tests`, with the experiments E7–E17. The CSV files go to `src/results/` and the figures are generated with `python src/plots.py`.

- **Optimizations.** Distances presorted per seller (pairs (c_{ij}, i)) to try the closest warehouses first; incremental evaluation of the cost by Δ in $O(1)$ per move; and dynamic programming for the knapsacks, which are solved thousands of times.
- **Correctness.** More than 4400 automated tests: feasibility and cost of every solution, the knapsack against enumeration, and every exact method against brute force on small random instances.
- **Unassigned sellers and feasibility.** Partial solutions and the penalty p are central for the heuristics on the tight instances: guided by the cost, the constructive heuristics leave thousands of sellers unassigned.
- **Difficulties.** (i) On the large instances the bound of the branch-and-price cannot be completed with an LP (E14), which led to the bundle method. (ii) On the large type-e instances the VND of the initial solution takes longer than the whole budget of the metaheuristics (E3, E5). (iii) Integrating the fixing inside the solver required care with several details of SCIP; one of them made solutions already found get lost, and the tests caught it.

6 Experiments

6.1 Environment

- **Processor / RAM / OS:** AMD Ryzen 9 8945HX / 32 GB / Linux.
- **Compiler and solvers:** g++ 16.2 (`-O2`); SCIP 10.1 and HiGHS 1.15.
- **Measurement:** `std::chrono` around each method. The heuristics and the local search are deterministic (one run); the metaheuristics are averaged over $R = 5$ seeds and stopped by time according to the size (2/10/60 s for $n \leq 200$ / $n \leq 900$ / larger). The exact methods have a limit of 600 s (300 s in E14–E16) and run several instances in parallel.
- **Reproducibility:** the experiments are automated in the programs `tests.cpp` and `exact_tests.cpp` of `src/`; they produce the CSV files of `src/results/` and the figures.

6.2 Instances

We use the five benchmark groups of the GAP (Table 1): a and b from the OR-Library [2], and c, d and e from Yagiura's site (those of types c and d with $n \leq 200$ are the OR-Library ones), plus the real ThunderPack instance. In the real one the total capacity is 15,235 and the total demand 10,707 (ratio 0.70); the demands range from 5 to 19 units and the capacities from 20 to 250. Appendix A gives the best bound and the best solution of each instance.

Table 1: Instance groups: size and number of instances with the optimum proven at the end of the work.

Group	Instances	m	n	Source	Optimum proven
a	6	5–20	100–200	OR-Library	6
b	6	5–20	100–200	OR-Library	6
c	15	5–80	100–1600	OR-Library and Yagiura	10
d	15	5–80	100–1600	OR-Library and Yagiura	2
e	15	5–80	100–1600	Yagiura	11
real	1	310	1100	ThunderPack	1
Total	58				36

6.3 Metrics

- **Cost** $f(s)$: objective function (4) (penalty included).
- **Unassigned**: number of sellers without a warehouse (measures feasibility, Q1).

- **Time** of execution (Q3).
- **True gap** $(f(s) - LB)/LB \cdot 100\%$ with respect to the *best lower bound* LB known for that instance (the best of all the relaxations and exact methods). If the best solution reaches LB, the optimum is proven.

6.4 Research questions

- **(RQ1)** Which constructive heuristic gives the best starting point (quality and % unassigned)?
- **(RQ2)** How much does each local search operator contribute, and first vs. best improvement?
- **(RQ3)** Does the VND improve over a single operator? Does the order of the neighborhoods matter?
- **(RQ4)** Which parameters of ILS (`strength`) and LNS (`destroy`) give the best quality/time trade-off, and how do both metaheuristics compare?
- **(RQ5)** Overall: constructive → VND → ILS/LNS, how much is gained and at what time cost, depending on the size of the instance?
- **(RQ6, real case)** What quality and time are obtained on the ThunderPack instance, and what does it imply for Q1–Q3?
- **(RQ7)** How far from the optimum are the heuristics, and how many instances do the exact methods solve?
- **(RQ8)** Is it worth starting the searches from the relaxations?
- **(RQ9)** Which bound is better, that of the cutting planes or the Lagrangian one? And which LP rounding?
- **(RQ10)** What limits the branch-and-price on the large instances?
- **(RQ11)** How much slack does the real network have?
- **(RQ12)** Can the instances still open be closed?

6.5 E1 — Constructive heuristics (RQ1)

Hypothesis. We expected the regret insertion to beat the nearest neighbor, by anticipating the assignments forced at the end, and the demand-based measures to help on the tight instances. The times of all four are negligible (at most 0.05 s).

Table 2: E1: mean true gap (%) and total unassigned sellers of each constructive heuristic, by group. NN: nearest neighbor; the other three are the regret insertion with $f_{ij} = c_{ij}$ (cost), d_{ij} (demand) and d_{ij}/b_i (relative demand).

		NN		Regret, cost		Regret, demand		Regret, rel. demand	
Group	Inst.	Gap	Unassigned	Gap	Unassigned	Gap	Unassigned	Gap	Unassigned
a	6	1.30	0	**0.01**	0	109.55	0	109.55	0
b	6	116.37	122	97.78	107	**92.12**	0	**92.12**	0
c	15	203.84	1,779	202.53	1,782	**109.15**	0	140.71	0
d	15	179.06	4,462	177.03	4,543	**67.74**	0	68.48	0
e	15	1,561.17	6,622	1,606.73	6,837	**692.45**	0	692.84	0
real	1	15.98	0	**5.65**	0	15.98	0	496.09	0

The results (Table 2) confirm the hypothesis only in part. The cost-based insertion wins in group *a* (loose) and on the real instance, but on the tight groups the two cost-guided constructive heuristics leave hundreds or thousands of sellers unassigned, and the penalty dominates the cost. The demand-based measures assign every seller, at a higher cost, and are the best option from *b* to *e*. Even so, the starting point is very far from the optimum: between 52 and 692 % on average in those groups.

6.6 E2 — Local search operators (RQ2)

Hypothesis. We expected *best improvement* to reduce the cost more per sweep than *first* (at a higher time cost) and the generalized operators to add something over the simple ones. Starting from three constructive heuristics (nearest neighbor, regret by cost and by demand) we applied each operator until a local optimum on the 58 instances.

Table 3: E2: improvement (%) over the cost of the constructive solution depending on which one it starts from, largest improvement, mean time and mean number of passes (these three over all the runs). The *multiple* operators apply a fixed strategy (q=1 . . . 10), with no first/best variant.

	Mean improvement (%) from					
Operator	NN	Regret, cost	Regret, demand	Max improvement (%)	Mean time (s)	Mean passes
relocate (first)	0.00	0.00	36.55	58.91	0.004	240.1
relocate (best)	0.00	0.00	37.74	56.05	0.010	78.7
swap (first)	1.70	0.70	**48.01**	79.10	0.143	614.2
swap (best)	1.34	0.62	38.35	55.00	0.270	82.6
relocate multiple	0.00	0.00	38.04	67.11	0.004	221.6
swap multiple	1.48	0.70	44.50	78.33	0.095	320.0

The result (Table 3) depends on the starting point. From the nearest neighbor and the cost-based insertion, no *relocate* improves, because those constructive heuristics already assign each seller to its cheapest feasible warehouse; all the work is done by the *swap*, and it is little. From the demand-based regret, which assigns every seller but at a high cost, all the operators improve a lot (between 36 and 48 %). In the *swap*, *first improvement* improves more than *best* (48.0 against 38.4 %) and in half the time: accepting right away leaves more room for the following moves. Used alone, the multiple operators stay close to the simple ones; their advantage shows up when they are combined in the VND (E3).

6.7 E3 — VND vs. single operators (RQ3)

Hypothesis. We expected the VND, by combining neighborhoods, to beat any single operator, above all on the tight instances. We start from the best constructive solution and compare simple *relocate* and *swap*, a VND that chains only those two, and the full VND.

Table 4: E3: mean true gap (%) of the final solution, from the best constructive solution, and time of the full VND (mean and maximum), by group.

Group	Constructive	Relocate	Swap	VND reloc.+swap	Full VND	Mean t (s)	Max t (s)
a	0.01	0.01	0.01	0.01	0.01	0.01	0.02
b	52.92	28.28	18.31	16.03	**3.01**	0.02	0.07
c	109.15	25.36	10.89	4.75	**3.73**	3.26	27.93
d	66.36	13.59	9.46	9.09	**5.10**	6.03	47.43
e	691.59	431.01	162.74	**160.33**	218.56	39.07	382.73
real	5.65	5.65	3.65	3.08	**2.34**	0.37	0.37

The data (Table 4) confirm the hypothesis in every group except *e*. The full VND lowers the mean gap from 53 to 3 % in *b*, from 109 to 4 % in *c* and from 66 to 5 % in *d*, and on the real instance from 718 to 695.5, because the ejection and the *3-cycle* place sellers that the moves between two warehouses cannot relocate. In *e*, instead, it ends up worse than the *swap* alone and takes much longer (up to 383 s): the order of the neighborhoods matters, because each one changes the point from which the others continue. Figure 2b shows how the time of the VND grows with the size.

6.8 E4 — Parameter tuning: ILS and LNS (RQ4)

As the stopping criterion we use time (not the number of iterations), scaled by size: 2 s for $n \leq 200$, 10 s for $n \leq 900$ and 60 s for the largest. Each configuration is averaged over 5 seeds. We tuned the `strength` of the ILS perturbation and the destruction size `destroy` of the LNS on three representative instances.

Hypothesis. ILS should be little sensitive to `strength` and LNS sensitive to `destroy`. We expected comparable performance from both.

Table 5: Tuning: mean cost (5 seeds) by parameter value. Best per row in bold. Time limit: 2 s (`b10200`), 10 s (`e15900`), 60 s (`real`). The optima are 2,827, 102,421 and 679.60.

ILS — perturbation strength `strength`				
Instance	5	10	20	40
`b10200`	2,845.0	2,843.0	2,841.2	**2,836.2**
`e15900`	420,850.0	420,454.2	419,350.0	**419,334.6**
`real`	687.36	684.04	683.62	**683.44**
LNS — destruction size `destroy`				
Instance	20	50	100	
`b10200`	**2,836.4**	2,849.2	2,871.0	
`e15900`	419,333.8	418,982.6	**417,722.6**	
`real`	686.80	682.80	**682.42**	

The hypothesis holds (Table 5). ILS barely improves when the `strength` is raised (40 is the best, but within one standard deviation of 20, which we kept), and LNS prefers to destroy little on `b10200` and a lot on the large ones; we use 20 for $n \leq 900$ and 100 for the rest. Both perform similarly. But on `e15900` both stay about four times above the optimum: the 10 s budget is not even enough for the VND of the initial solution (29 s in E3), and there the configuration does not matter.

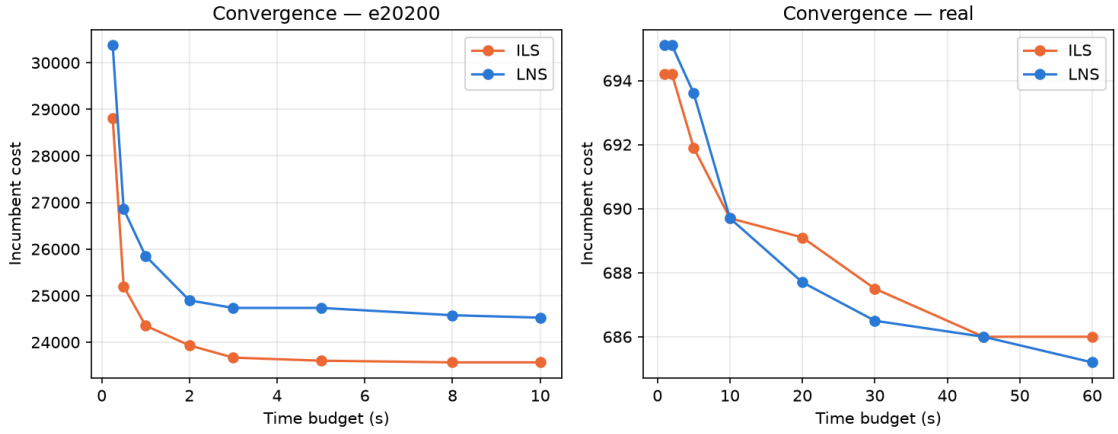


Figure 1: Convergence: incumbent cost vs. time budget, for ILS and LNS (seed 1), on a tight instance (`e20200`, optimum 22,379) and on the real one (optimum 679.60).

6.9 E5 — Final comparison (RQ5)

Hypothesis. We expected quality to grow monotonically, constructive < VND < metaheuristic, with larger gains on the tight instances, and LNS to make the difference precisely there. Table 6 reports the mean true gap of each method by group.

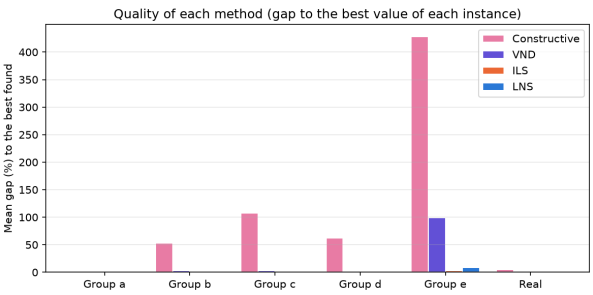
Table 6: Final comparison: mean true gap (%) by method and group (ILS and LNS: mean cost over 5 seeds). Mean time of the metaheuristic (limit according to size).

Group	Constructive	VND	ILS	LNS	MH time (s)
a (avg.)	0.01	0.01	**0.00**	0.01	2.0
b (avg.)	52.92	3.01	0.33	**0.31**	2.0
c (avg.)	109.15	3.73	0.99	**0.98**	16.9
d (avg.)	66.36	5.10	**2.85**	2.98	16.8
e (avg.)	691.59	218.56	**119.69**	125.59	16.8
real	5.65	2.34	0.83	**0.81**	60.0

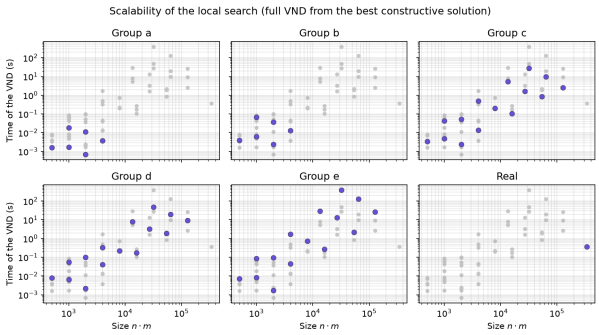
Table 7: ILS vs. LNS: cost over 5 seeds (best / worst / mean / standard deviation) on one instance per group plus the real one, and the optimum. Lower is better; best mean cost per row in bold.

		ILS				LNS			
Instance	Optimum	Best	Worst	Mean	Std. dev.	Best	Worst	Mean	Std. dev.
a05100	1,698	1,698	1,698	**1,698**	0.00	1,698	1,698	**1,698**	0.00
b10200	2,827	2,831	2,849	2,841.20	6.01	2,833	2,839	**2,836.20**	2.32
e15900	102,421	423,256	423,279	423,263.20	8.47	423,216	423,272	**423,254.40**	20.49
real	679.60	683.70	686.90	685.26	1.17	683.70	686.90	**685.12**	1.06

The progression constructive → VND → MH is monotonic in every group, but LNS does not make the difference: ILS and LNS are on par (ILS has the better mean cost on 30 instances and LNS on 22; Table 7). In *a*, *b*, *c* and the real instance both stay below 1 % from the optimum; in *d*, around 3 %; and in *e*, very far: on several of the large instances the VND of the initial solution takes longer than the whole budget, and on the rest they stall. Without a lower bound these distances could not be seen: against the best value found, the mean gap of the metaheuristics did not exceed 9 % in any group (Figure 2a). Starting from the relaxations corrects this (E9).



(a) Mean gap by method and group, against the best value found in E5.



(b) Time of the VND vs. size $n \cdot m$ (log-log).

Figure 2: Summary of the final comparison: quality (gap) and scalability of the local search.

6.10 E6 — Real case: ThunderPack (RQ6)

Business reading (Q1–Q3). The real instance is solved with 0 unassigned sellers by every method: the aggregate capacity (ratio 0.70) is enough and the geography does not prevent it, so the network does not need to be expanded for the current demand (**Q1**; E15 measures how much slack remains). The mean distance per seller goes down from 0.65 (constructive) to 0.62 at the optimum: close and reasonable assignments (**Q2**). In time (**Q3**), the constructive heuristic is instantaneous and the VND already cuts 3 % in 0.15 s; the Lagrangian heuristic reaches 679.8 in

Table 8: Real instance ($m = 310$, $n = 1100$): cost, unassigned sellers, mean distance per assigned seller and time, by method. Top, the heuristic chain (ILS and LNS: best of 5 seeds, and mean cost in parentheses); bottom, the methods based on the relaxations.

Method	Cost	Unassigned	Mean distance	Time (s)
Best constructive	718.0	0	0.65	< 0.01
+ VND	695.5	0	0.63	0.15
+ ILS	682.3 (683.4)	0	0.62	60.0
+ LNS	682.0 (682.4)	0	0.62	60.0
Lagrangian heuristic + VND	679.8	0	0.62	≈ 3
MIP-LNS	**679.6** (679.6)	0	0.62	60.0
SCIP with fixing (optimum proven)	**679.6**	0	0.62	8.8

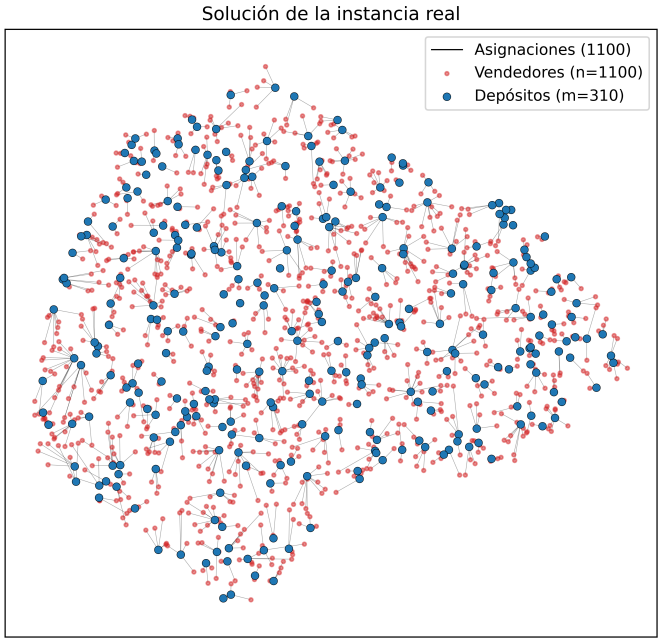


Figure 3: Geographic layout of the real instance ($n = 1100$ sellers in red, $m = 310$ warehouses in blue). The gray lines join each seller with its warehouse in a heuristic solution of cost 681.10.

about 3 s, 0.03 % from the optimum, and SCIP with fixing proves the optimum, 679.60, in 8.8 s. A one-hour run of the heuristic chain had reached 681.10, 0.22 % above it. An interactive tool can therefore deliver the optimal solution in seconds.

6.11 Worst and best case (synthetic instances)

To see in which cases each method fails or succeeds, besides the benchmark we built minimal *ad hoc* instances that trigger the worst and the best case of each kind of method. Table 9 summarizes what we observed.

In the *myopic greedy* case, the nearest neighbor assigns the first seller to a warehouse that the second one needed and ends at 101, while the regret solves it at 3. In the *3-cycle local optimum* we build an assignment that no *relocate multiple* or *swap multiple* (of two warehouses) can leave, but the 3-warehouse *cyclic exchange* of the VND can, and this justifies that operator. Finally, the metaheuristic recovers the optimum from a very bad constructive solution, but on an already optimal instance it cannot improve: what it gains depends on how much room the local search leaves.

Table 9: Worst/best case on synthetic instances. In each case, the cost that each method obtains on the same instance.

Case	Method	Cost	Interpretation
Myopic greedy	Nearest neighbor	101	*worst case* of the greedy
	Regret insertion	3	*best case* of the regret
3-cycle local optimum	relocate + swap multiple	30	stuck (*worst case* of 2 warehouses)
	Full VND (cycle3)	3	escapes (*best case* of the VND)
Metaheuristic	ILS from a bad constructive	3	recovers the optimum (*best case*)
	ILS on an optimal instance	2	only *overhead* (*worst case*)

6.12 E7–E10 — Bounds, exact methods and true gaps (RQ7, RQ8)

Hypothesis. We expected the Lagrangian bound to be considerably better than the LP one, the heuristics based on the relaxations to give good starting points, and the fixing to speed up SCIP a lot, to the point of closing most instances in 600 s.

Table 10: E7: mean distance (%) of the LP bound and of the Lagrangian bound to the best known bound, instances solved to optimality in 600 s by SCIP, by SCIP with fixing and by the branch-and-price with fixing (B&P), and mean percentage of seller-warehouse pairs discarded by the fixing.

		Bound gap (%)		Solved in 600 s			
Group	Inst.	LP	Lagrangian	SCIP	SCIP + fix.	B&P + fix.	% discarded
a	6	0.047	0.000	6	6	6	0.0
b	6	0.483	0.063	6	6	6	32.9
c	15	0.353	0.040	9	9	7	63.8
d	15	0.095	0.022	1	0	2	0.0
e	15	0.128	0.018	9	8	7	25.2
real	1	2.855	0.040	1	1	0	98.8
Total	58			32	30	28	

Bounds and exact methods (E7). The bounds behave as we expected (Table 10): the Lagrangian bound stays on average within 0.07 % of the best bound in every group, several times closer than the LP one (on the real instance, 679.33 against 660.20). The fixing, instead, depends on the starting solution. On the real instance it discards 98.8 % of the pairs and SCIP goes from 88.6 to 8.8 s. In *c* and *e* it discards between a quarter and two thirds. In *d* it discards nothing, even though the starting solution is not farther from the bound than in *c* or *e* (between 0.2 and 1.8 %): on that type of instances a solution much closer to the optimum is needed. For example, on `d05100`, with a solution of 6388 it discards nothing, with 6360 it discards 34 % and with the optimum, 6353, 64 %. In group *a* the 0 % has another cause: the bound already proves that the starting solution is optimal. Together, the three methods solve 33 instances in 600 s; type *d* remains almost entirely open.

Matheuristic (E8, E9). To configure the MIP-LNS we tried 27 combinations of starting point, destruction method and size on five instances (E8): the starting point weighed much more than the rest. From the Lagrangian heuristic the mean gap stayed between 0.01 and 1.1 % on all five; from the best constructive solution it reached 234 % on `e15900`. We chose to start from the Lagrangian heuristic and to destroy groups of nearby warehouses. In E9 we compared, with the same budget, the LNS of E5, the same LNS started from the Lagrangian heuristic and the MIP-

LNS (Figure 4). Almost all the gain comes from the start: the LNS started from the Lagrangian heuristic lowers the mean gap in e from 125.6 to 0.93 % and on the real instance from 0.81 to 0.015 %. The MIP-LNS adds a smaller improvement: it is the best in b, e and the real one, where it finds the optimum with 4 of the 5 seeds, but in d it falls slightly behind.

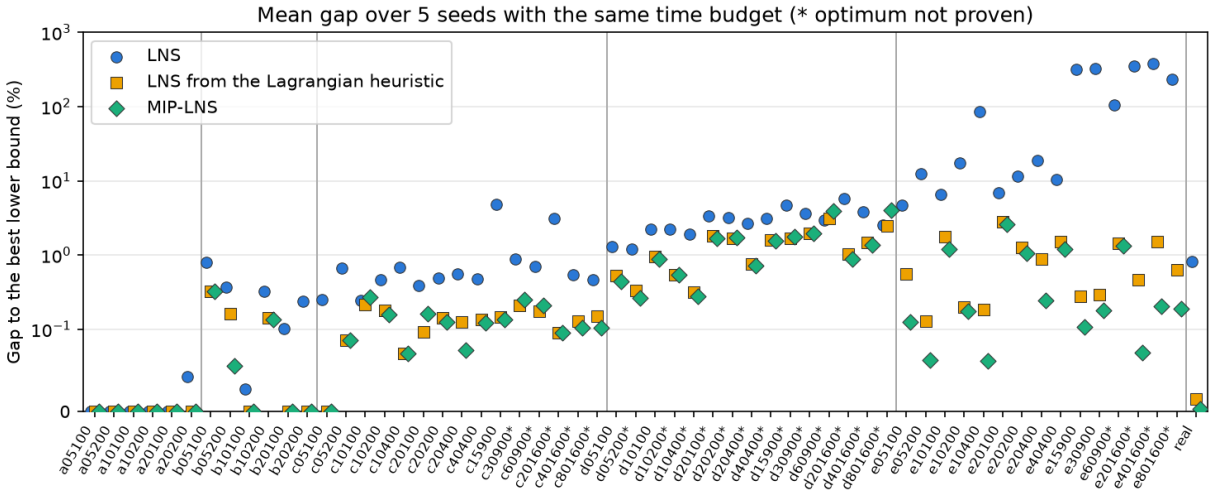


Figure 4: E9: mean true gap (5 seeds, same budget) of the LNS, of the LNS started from the Lagrangian heuristic and of the MIP-LNS, on the 58 instances (log scale; $*$: optimum not proven).

Table 11: E10: mean true gap (%) of each method, by group, against the best known lower bound. In the last row, instances with the optimum proven at the end of the work.

Method	a	b	c	d	e	real	All
Best constructive	0.01	52.92	109.15	66.36	691.59	5.65	229.82
+ VND	0.01	3.01	3.73	5.10	218.56	2.34	59.16
ILS (mean)	0.00	0.33	0.99	2.85	119.69	0.83	31.99
LNS (mean)	0.01	0.31	0.98	2.98	125.59	0.81	33.55
LP rounding + VND	0.00	0.36	0.55	1.02	2.52	1.75	1.13
Lagrangian heuristic + VND	0.00	0.14	0.25	0.99	0.95	0.03	0.58
LNS from the Lagrangian heuristic (mean)	0.00	0.10	0.13	1.35	0.93	0.01	0.63
MIP-LNS (mean)	0.00	0.09	0.13	1.46	0.59	0.00	0.57
SCIP + fixing (600 s)	0.00	0.00	0.06	0.58	0.42	0.00	0.27
Branch-and-price + fixing (600 s)	0.00	0.00	0.08	0.50	0.26	0.00	0.22
Optimum proven	6/6	6/6	10/15	2/15	11/15	1/1	36/58

True gaps (E10). Table 11 sums everything up. The metaheuristics of the first stage stay within 1 % in a, b, c and the real instance, but not in d or e. The heuristics based on the relaxations are much more even: the Lagrangian heuristic with VND, alone and in a few seconds, stays on average at 0.58 %, against 33.6 % for the LNS. The exact methods, stopped at 600 s, end within 0.6 % in every group. With all the experiments, the optimum is proven on 36 of the 58 instances.

6.13 E11–E12 — Cutting planes and roundings (RQ9)

Hypothesis. We expected the cutting planes to bring the LP bound closer to the Lagrangian one, and the rounding of Shmoys and Tardos, which has a cost guarantee, to give better solutions than the simple rounding.

The first part holds only halfway. The cover inequalities close between 30 and 89 % of the distance between the LP bound and the Lagrangian one, depending on the group, always less than the cuts that SCIP adds on its own. The exact separation reaches the Lagrangian bound on almost every instance of b and e and on the real one, but it takes up to the 600 s of the limit and in d it never finishes. The Lagrangian bound, instead, is computed in less than a minute (E14): for this problem it is better than any family of cuts. The second part does not hold: the guarantee of Shmoys and Tardos is verified on the 58 instances, but their assignment exceeds the capacity of many warehouses (112 of 310 on the real one) and, after it is repaired, it is better than the simple rounding on 25 instances and worse on 32. The simple rounding is enough.

6.14 E13–E14 — The branch-and-price (RQ10)

Hypothesis. We expected the speed-up techniques of the branch-and-price to help, and a better solver for the linear program of the root to let it advance on the large instances.

On eight small instances that the branch-and-price solves, switching off each technique one at a time does not change how many it solves, and the version without any of them is the fastest (15 s against 30 s; E13): on instances of this size, the techniques cost more than they save. The problem lies in the large ones (E14, Figure 5): neither the simplex nor the interior point method of HiGHS finishes the linear program of the root in 300 s on four of eight instances, the real one included. The bundle method, instead, computes the same bound on 57 of the 58 instances in a median of half a second and in at most 38 s (on the remaining one it stays 0.004 from the exact value). Even so, the branch-and-price needs the solution of the linear program to branch, so on the instances with 900 sellers or more it barely gets past the root: on almost all of them it explores one or two nodes in 600 s (E17).

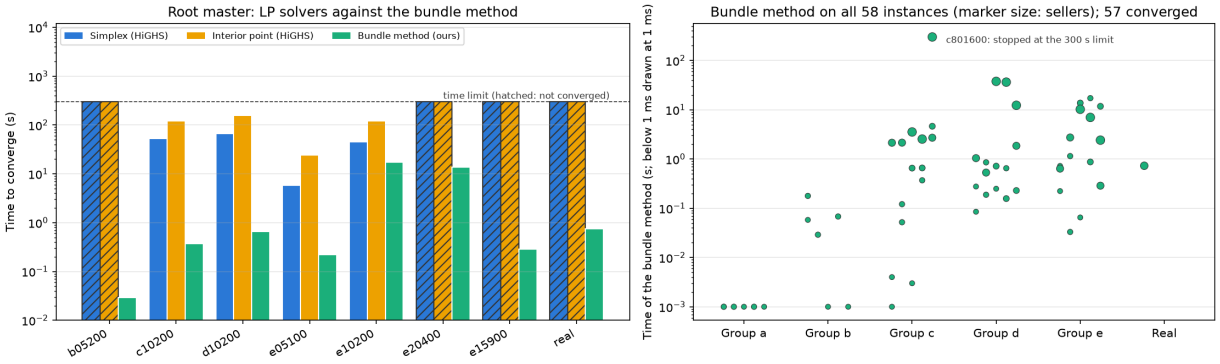


Figure 5: E14: left, time to compute the bound of the root with the simplex, the interior point method and the bundle method (hatched: did not finish in 300 s); right, time of the bundle method on the 58 instances, by group (marker size: number of sellers).

6.15 E15 — Slack of the real network (RQ11)

Hypothesis. We expected the shadow prices of the capacities (how much the cost goes down per unit of extra capacity, according to the LP) to estimate well the saving of expanding a warehouse, and closing a heavily loaded one to cost a lot.

Starting from the optimum, we solved the real instance again in three kinds of scenarios: 10 more units of capacity in each of the 15 warehouses with the largest shadow price, the closure of each of the 15 most loaded warehouses and an increase of the demand (Figure 6).

The shadow price estimates the saving well: between 0.9 and 2.3 per 10 units, at most 0.34 % of the cost, so expanding one warehouse does not change the picture. Closing one of the most loaded warehouses costs between 2.4 and 20.2 (between 0.4 and 3 %), always without leaving sellers out; the most critical one is warehouse 239, whose closure costs 20.2 although its shadow

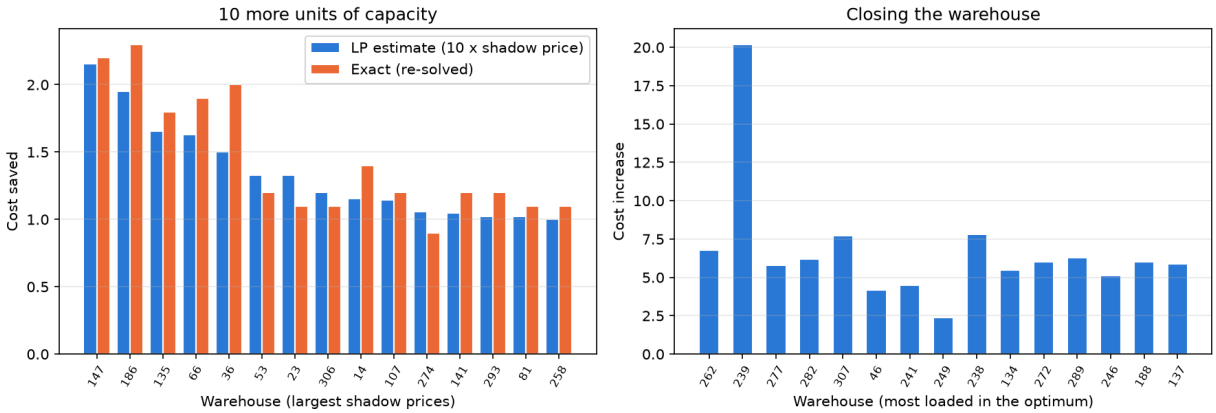


Figure 6: E15: left, saving of 10 more units of capacity in the 15 warehouses with the largest shadow price, estimated by the LP and exact; right, cost increase when closing each of the 15 most loaded warehouses.

price is only 0.08, so the shadow price does not anticipate the cost of a closure. For the demand we multiplied each one by 1.05, 1.10 and 1.20, rounding up; since the demands are small integers, the total demand actually grows by 10, 14 and 24 %. Even so, every seller fits, with costs of 721.9, 744.9 and 818.4.

6.16 E16–E17 — The open instances (RQ12)

Hypothesis. Since the fixing did not work on d, we expected that starting from better solutions and re-fixing during the search would close a good number of the open instances.

In E16, on the 30 instances that the fixing of E7 does not close in 100 s, re-fixing doubled the instances solved in 300 s (from 2 to 4), and starting from the MIP-LNS gave the lowest mean gap (0.33 %), but no variant solved more than 4. On the large type-d instances, 60 s of MIP-LNS even gave worse solutions than those of E7. That is why in E17 we tried, on the 24 instances still open, four heuristics with 120 s (Table 12) and, from the solution of the portfolio, SCIP and the branch-and-price with re-fixing for 600 s.

Table 12: E17: mean true gap (%) of each heuristic with 120 s on the 24 open instances, by group, and instances on which it is the best (ties included).

Heuristic (120 s)	c (5)	d (13)	e (6)	All	Best on
MIP-LNS	0.149	0.698	**0.122**	0.440	9
Tabu search	0.141	1.323	1.390	1.094	0
LP rounding + MIP-LNS	0.153	0.478	0.223	0.346	4
Portfolio	**0.099**	**0.426**	0.148	**0.288**	12

The hypothesis holds only in part. The portfolio was the best heuristic on average, and with E17 the best known solution went down on 18 of the 24 instances. With that, the fixing started to work where the solution ended up very close to the optimum: SCIP with re-fixing closed `e40400` and `e30900`, while the discarded pairs went up from 45 to 90 % and from 51 to 95 %. In type d it only worked on the two instances that ended up closest to the optimum (`d05200` and `d20100`, 0.05 % from the bound, where it discarded 42 and 50 % of the pairs); on the others, the remaining distance (between 0.1 and 0.9 %) is still too large. 22 instances remain open (5 of c, 13 of d and 4 of e), all with a gap below 1 %.

7 Conclusions

- **Answer to Q1–Q3 (real case).** The optimum of the real instance is 679.60, with every seller assigned: the current network is enough (**Q1**) and has slack to spare, since it absorbs 24 % more demand without leaving anyone out and the closure of any of the 15 most loaded warehouses costs at most 3 % (E15). The mean distance is 0.62 per seller (**Q2**). The Lagrangian heuristic gives, in about 3 s, a solution 0.03 % from the optimum, and SCIP with fixing proves the optimum in 8.8 s (**Q3**): such a tool can be used in real time.
- **Measuring against a bound changes the reading.** Compared only with each other, the metaheuristics did not exceed a 9 % mean gap in any group; against the lower bound they stay around 3 % from the optimum in d and above 100 % in e.
- **The relaxations give the best starting points.** The Lagrangian heuristic with VND stays, alone, at 0.58 % on average, and the LNS or the MIP-LNS started from it, within 1 % in every group except d. The start weighs more than the rest of the configuration.
- **The Lagrangian bound is the central tool.** It is better than the LP bound and than that of the cutting planes, and it allows discarding almost the whole problem when the known solution is close to the optimum (on the real instance, 98.8 % of the pairs).
- **What remains open.** Type d, with very tight capacities, is by far the hardest: 13 of its 15 instances are still open, all with a gap below 1 %. The next steps would be to let the branch-and-price branch without solving the linear program of the root, and a more complete tabu search for that type of instances.

References

[1] P. Avella, M. Boccia and I. Vasilyev. A computational study of exact knapsack separation for the generalized assignment problem. *Computational Optimization and Applications*, 45(3):543–555, 2010.

[2] J. E. Beasley. OR-Library: distributing test problems by electronic mail. *Journal of the Operational Research Society*, 41(11):1069–1072, 1990.

[3] J. F. Benders and J. A. E. E. van Nunen. A property of assignment type mixed integer linear programming problems. *Operations Research Letters*, 2(2):47–52, 1983.

[4] O. Briant, C. Lemaréchal, Ph. Meurdesoif, S. Michel, N. Perrot and F. Vanderbeck. Comparison of bundle and classical column generation. *Mathematical Programming*, 113(2):299–344, 2008.

[5] M. L. Fisher, R. Jaikumar and L. N. Van Wassenhove. A multiplier adjustment method for the generalized assignment problem. *Management Science*, 32(9):1095–1103, 1986.

[6] Q. Huangfu and J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018. (HiGHS, `https://highs.dev`).

[7] S. Martello and P. Toth. An algorithm for the generalized assignment problem. In J. P. Brans (ed.), *Operational Research '81*, 589–603. North-Holland, 1981.

[8] M. Posta, J. A. Ferland and P. Michelon. An exact method with variable fixing for solving the generalized assignment problem. *Computational Optimization and Applications*, 52(3):629–644, 2012.

[9] M. Savelsbergh. A branch-and-price algorithm for the generalized assignment problem. *Operations Research*, 45(6):831–841, 1997.

[10] SCIP Optimization Suite, version 10.1. `https://www.scipopt.org`.

[11] D. B. Shmoys and É. Tardos. An approximation algorithm for the generalized assignment problem. *Mathematical Programming*, 62:461–474, 1993.

[12] M. Yagiura, T. Ibaraki and F. Glover. An ejection chain approach for the generalized assignment problem. *INFORMS Journal on Computing*, 16(2):133–151, 2004.

A Results per instance

Table 13: Best lower bound and best solution of each instance at the end of the work, whether the optimum is proven, its gap (%) and the mean true gap (5 seeds) of the LNS (E5) and of the MIP-LNS (E9).

Instance	m	n	Lower bound	Best solution	Optimum	Gap (%)	LNS (%)	MIP-LNS (%)
a05100	5	100	1,698	1,698	yes	0.000	0.00	0.00
a05200	5	200	3,235	3,235	yes	0.000	0.00	0.00
a10100	10	100	1,360	1,360	yes	0.000	0.00	0.00
a10200	10	200	2,623	2,623	yes	0.000	0.00	0.00
a20100	20	100	1,158	1,158	yes	0.000	0.00	0.00
a20200	20	200	2,339	2,339	yes	0.000	0.04	0.00
b05100	5	100	1,843	1,843	yes	0.000	0.79	0.33
b05200	5	200	3,552	3,552	yes	0.000	0.37	0.06
b10100	10	100	1,407	1,407	yes	0.000	0.03	0.00
b10200	10	200	2,827	2,827	yes	0.000	0.33	0.13
b20100	20	100	1,166	1,166	yes	0.000	0.10	0.00
b20200	20	200	2,339	2,339	yes	0.000	0.24	0.00
c05100	5	100	1,931	1,931	yes	0.000	0.25	0.00
c05200	5	200	3,456	3,456	yes	0.000	0.66	0.09
c10100	10	100	1,402	1,402	yes	0.000	0.24	0.27
c10200	10	200	2,806	2,806	yes	0.000	0.46	0.16
c10400	10	400	5,597	5,597	yes	0.000	0.69	0.07
c20100	20	100	1,243	1,243	yes	0.000	0.39	0.16
c20200	20	200	2,391	2,391	yes	0.000	0.48	0.12
c20400	20	400	4,782	4,782	yes	0.000	0.56	0.07
c40400	40	400	4,244	4,244	yes	0.000	0.47	0.12
c15900	15	900	11,340	11,340	yes	0.000	4.78	0.14
c30900	30	900	9,982	9,985	no	0.030	0.87	0.25
c60900	60	900	9,325	9,334	no	0.097	0.69	0.21
c201600	20	1600	18,802	18,807	no	0.027	3.10	0.10
c401600	40	1600	17,144	17,153	no	0.052	0.55	0.11
c801600	80	1600	16,284	16,296	no	0.074	0.46	0.11
d05100	5	100	6,353	6,353	yes	0.000	1.30	0.44
d05200	5	200	12,741	12,747	no	0.047	1.22	0.26
d10100	10	100	6,347	6,347	yes	0.000	2.23	0.89
d10200	10	200	12,428	12,444	no	0.129	2.23	0.54
d10400	10	400	24,959	24,983	no	0.096	1.93	0.28
d20100	20	100	6,182	6,185	no	0.049	3.33	1.67
d20200	20	200	12,232	12,312	no	0.654	3.21	1.72
d20400	20	400	24,561	24,638	no	0.314	2.67	0.71
d40400	40	400	24,350	24,578	no	0.936	3.15	1.54
d15900	15	900	55,403	55,476	no	0.132	4.64	1.77
d30900	30	900	54,833	54,983	no	0.274	3.63	1.96
d60900	60	900	54,551	54,884	no	0.610	2.96	3.88
d201600	20	1600	97,823	97,923	no	0.102	5.77	0.89
d401600	40	1600	97,105	97,450	no	0.355	3.82	1.35
d801600	80	1600	97,034	97,579	no	0.562	2.56	3.98
e05100	5	100	12,681	12,681	yes	0.000	4.72	0.13
e05200	5	200	24,930	24,930	yes	0.000	12.58	0.06
e10100	10	100	11,577	11,577	yes	0.000	6.55	1.20
e10200	10	200	23,307	23,307	yes	0.000	17.16	0.17
e10400	10	400	45,746	45,746	yes	0.000	85.18	0.06
e20100	20	100	8,436	8,436	yes	0.000	6.91	2.62
e20200	20	200	22,379	22,379	yes	0.000	11.44	1.07
e20400	20	400	44,877	44,877	yes	0.000	19.01	0.24
e40400	40	400	44,561	44,561	yes	0.000	10.52	1.20
e15900	15	900	102,421	102,421	yes	0.000	313.25	0.11
e30900	30	900	100,427	100,427	yes	0.000	324.31	0.18
e60900	60	900	100,147	100,331	no	0.184	105.37	1.33
e201600	20	1600	180,644	180,727	no	0.046	351.54	0.07
e401600	40	1600	178,293	178,516	no	0.125	380.77	0.20
e801600	80	1600	176,819	177,059	no	0.136	234.58	0.19
real	310	1100	679.60	679.60	yes	0.000	0.81	0.00